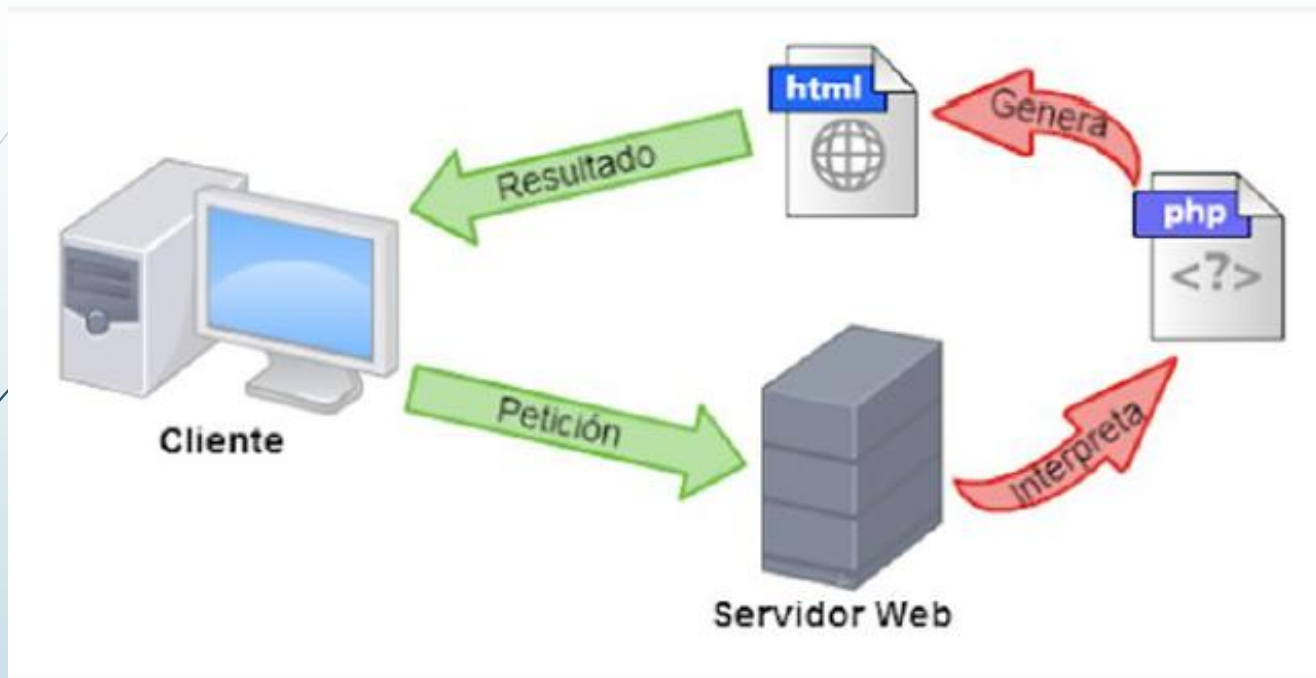


Desarrollo web en entorno servidor

PHP



Funcionamiento



PHP - básicos

- Separación de instrucciones ;
- Fichero .php `<?php ... ?>`
- No obligatorio cierre ?>
- Embebido en html

Comentarios

- De una sola línea

//

- Líneas múltiples

/* */

- De consola Unix

#

Mostrar por pantalla

- Mostrar una o más cadenas (ligeramente más rápido)

echo()

```
echo ( string $arg1 [, string $... ] ) : void
```

Concatenador

- Mostrar una cadena (devuelve 1)

print()

```
print ( string $arg ) : int
```

Tipos de datos escalares

- Integer

- Número entero
- Decimal, Octal, Hexadecimal, Binario

```
integer: 321
```

- Float

- Número real
- Enteros, racionales, irracionales

```
float: 321.0
```

Tipos de datos escalares

- String

- Serie de caracteres

```
string: "321"
```

- Boolean

- Puede ser TRUE/FALSE
- Con echo aparece 1 o nada

```
Boolean: True or False
```

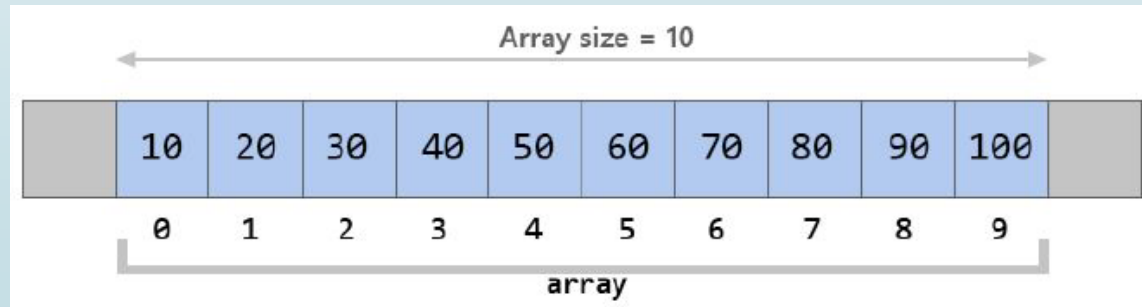
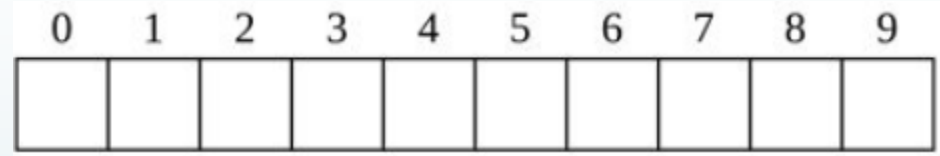
Tipos de datos compuestos

- Array

- Vector, matriz

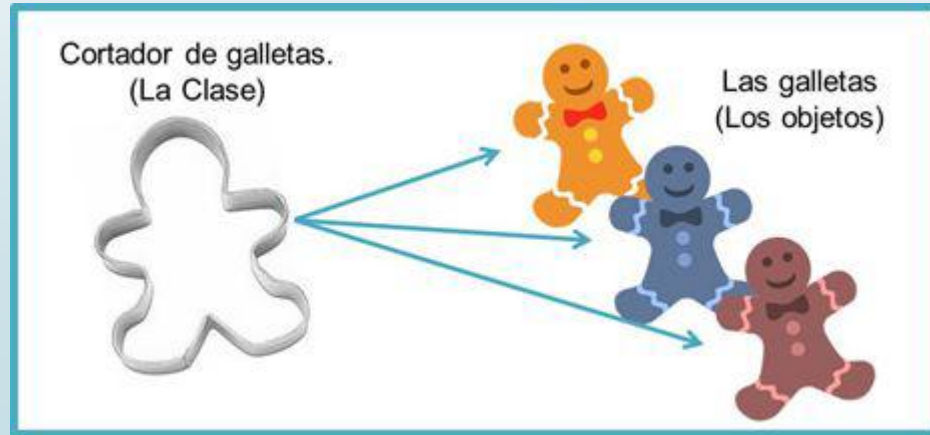
- Conjunto de valores que se almacenan juntos

- Accesibles por su posición (clave - valor)



Tipos de datos compuestos

- Object
 - Instancia de una clase
 - Ente con propiedades y métodos



PERRO

Raza
tamaño
Años
Color

Comer()
Dormir()
Sentar()
Correr()

Raza = Mastín napolitano
Tamaño = Grande
Edad = 5 años
Color = negro



Raza = maltés
Tamaño = Pequeño
Edad = 2 años
Color = Blanco

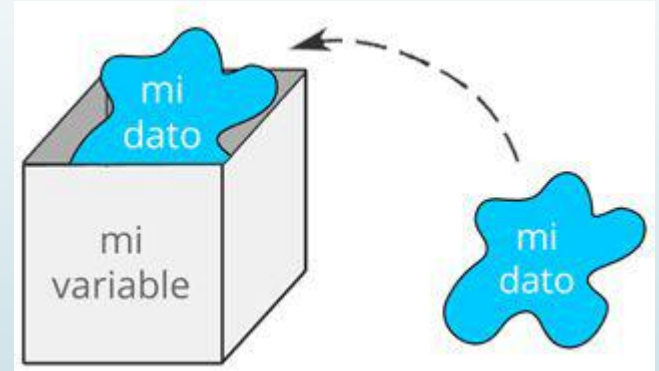


Raza = Chow Chow
Tamaño = Medio
Edad = 3 años
Color = Marrón



Variables

- Espacio en memoria y nombre asociado a ese espacio
 - Contiene información
 - Puede cambiar su valor
 - CaseSensitive



Variables

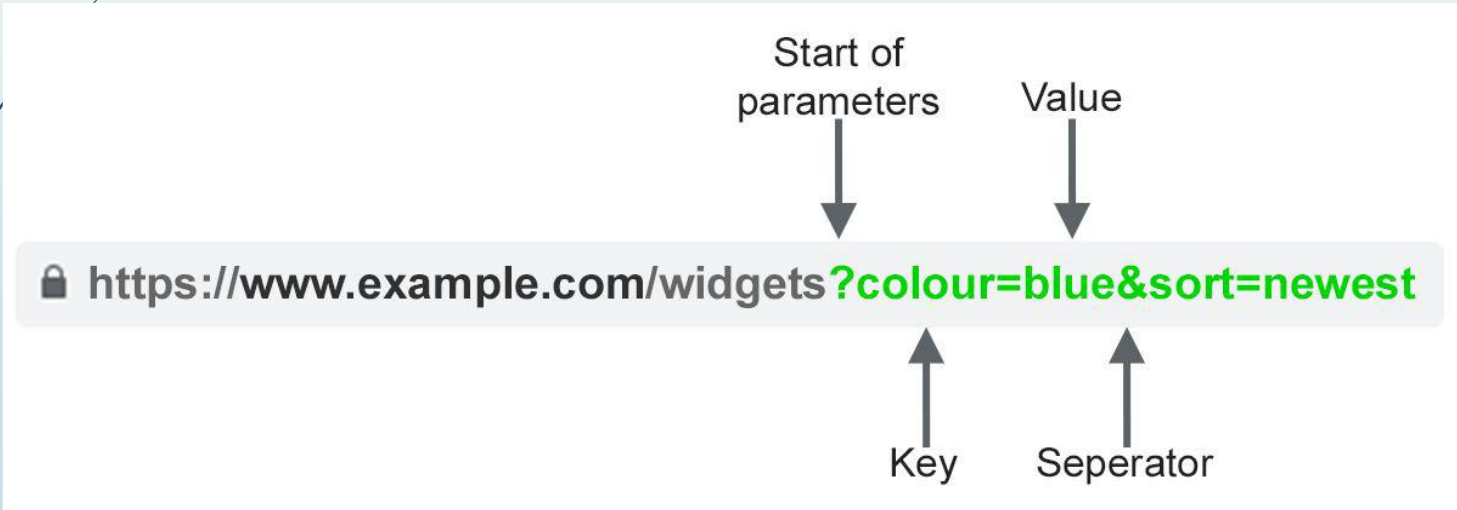
- Espacio en memoria y nombre asociado a ese espacio
 - Comienza por \$ `$Nombre_variable = Valor;`
 - Nombre debe comenzar
 - Letra
 - Carácter subrayado
 - No usar palabras reservadas (por confusión)

Variables ámbito

- Contexto donde se define la variable
 - Global
 - Fuera de una función
 - Local
 - A una función

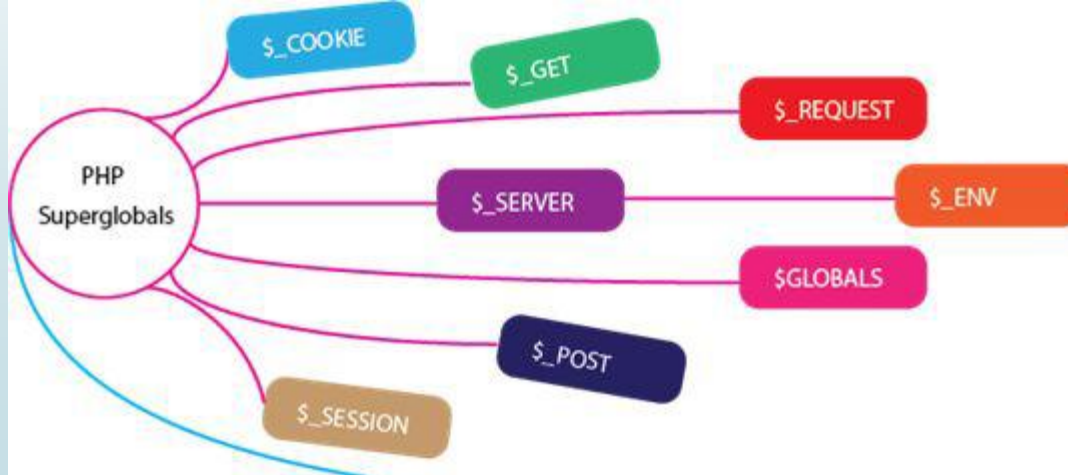
Variables paso por URL

- Pasar variables de una página a otra
- Pertenece al protocolo HTTP



Variables predefinidas

- Ya definidas por el lenguaje
- Algunas superglobales disponibles en todos los ámbitos



Constantes

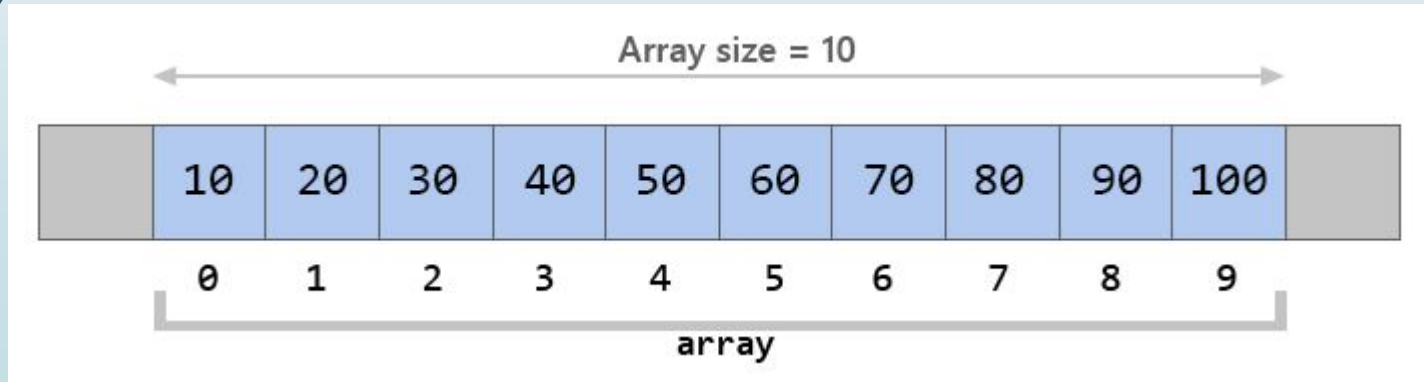
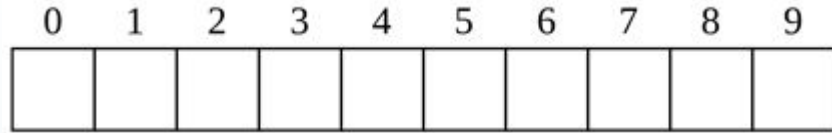
- No varía su valor
- Por convenio el nombre en mayúsculas
- Palabra reservada const
- Algunas predefinidas (Ej: PHP_VERSION)

KEYWORD	CONSTANT NAME	CONSTANT VALUE
const	NAME	= value;
BUILT-IN FUNCTION	CONSTANT NAME	CONSTANT VALUE
define	('NAME'	,value);

Arrays conceptos

- Array

- Vector, matriz
- Conjunto de valores agrupados
- Accesibles por su posición (clave - valor)



Arrays sintaxis

- Creación

- Constructor array() `$array_berria = array();`

- Sintaxis corta con corchete []

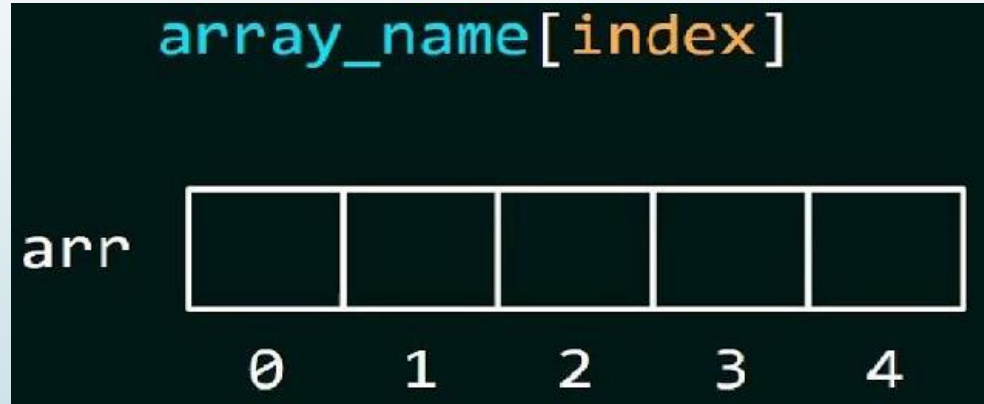
- `$array_indexatua = [12, 2, 34, 556, 67, 7];`

- Clave opcional

- Puede ser integer o string

Arrays accesso

- Accesso
 - `Nome_array[key]`



```
echo $array_indexatua[0];
```

Arrays acceso

- var_dump
 - Muestra el tipo de variable y su contenido
 - De manera estructurada

```
var_dump ( mixed $expression [, mixed $... ] ) : void
```

```
$array_indexatua = [12, 2, 34, 556, 67, 7];  
var_dump ($array_indexatua);
```

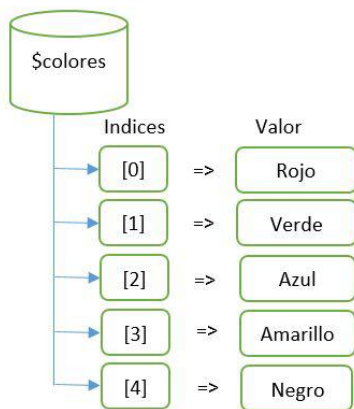
```
var_dump ($array_indexatua)  
array(6) { [0]=> int(12) [1]=> int(2) [2]=> int(34) [3]=> int(556) [4]=> int(67) [5]=> int(7) }
```

Arrays asociativos

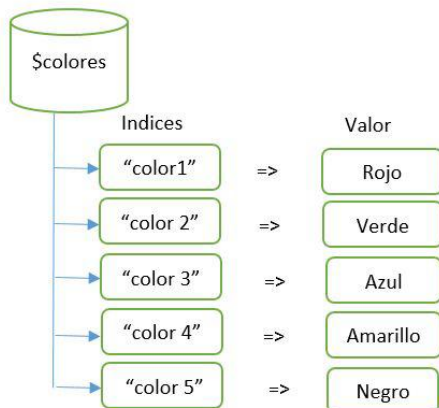
- Arrays cuyos keys son Strings personalizados
- Los Strings son CaseSensitive
- Aconsejable uso de comillas

```
$animales[perro]  
$animales['gato']
```

Array Indexados:



Array Asociativos:



```
$kalifikazioa = ['Ane'=> 9,  
                 'Jon'=> 5,  
                 'Ekain'=>8];
```

Operadores aritmeticos

- Toma varios valores y produce otro valor

ARITHMETIC OPERATORS		
operator name	operator	operand
addition	+	$\$a + \b
subtraction	-	$\$a - \b
multiplication	*	$\$a * \b
division	/	$\$a / \b
modulus	%	$\$a \% \b
exponentiation(a^b)	**	$\$a ** \b

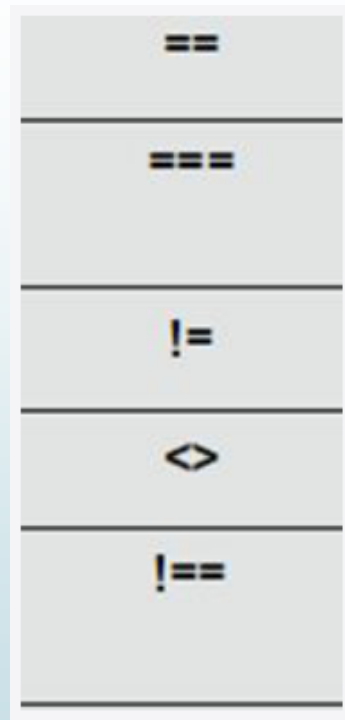
Operadores de asignación

- El operando de la izquierda se establece con el valor de la derecha
 - Básico el símbolo =

Operador de asignación	Expresión abreviada
$a+=b$	$a = a + b$
$a-=b$	$a = a - b$
$a*=b$	$a = a * b$
$a/=b$	$a = a / b$
$a\%=b$	$a = a \% b$

Operadores de comparación

- Compara dos valores
- Devuelve TRUE/FALSE
- Idéntico
 - Igual y del mismo tipo



Operadores de incremento/ Decremento

- Pre-incremento
 - Incrementa y devuelve el valor
- Post-incremento
 - Devuelve el valor e incrementa

`++$a`

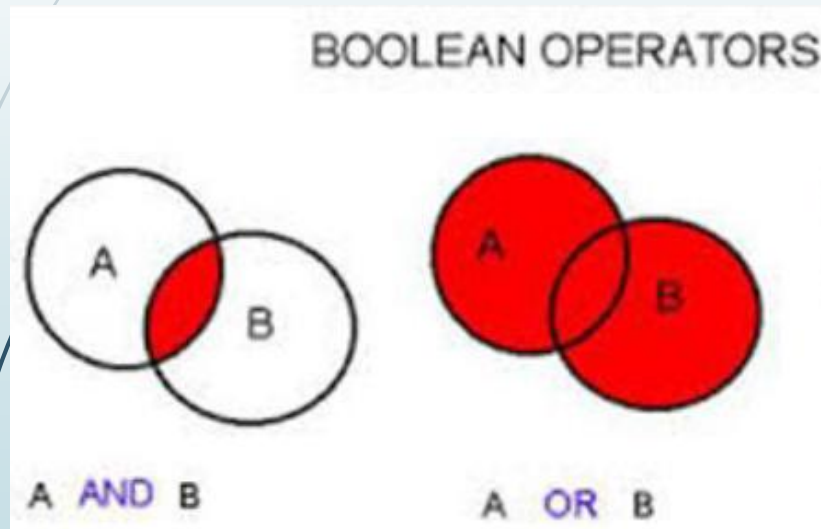
`$a++`

`--$a`

`$a--`

Operadores Lógicos

- Comprueban una o varias condición
- Devuelven TRUE/FALSE



`$a and $b`

`$a or $b`

`$a xor $b`

`! $a`

`$a && $b`

`$a || $b`

Operadores lógicos

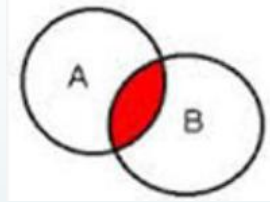
- AND

\$a and \$b

\$a && \$b

AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1



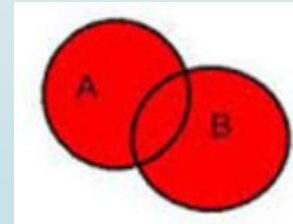
- OR

\$a or \$b

\$a || \$b

OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



Operadores lógicos

- XOR

\$a or \$b

XOR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0



- NOT

! \$a

NOT

x	F
0	1
1	0



Operadores para Strings

- Concatenador

- .

```
echo $array_indexatua[0] . "String-ak kateatzeko egokia da";
```

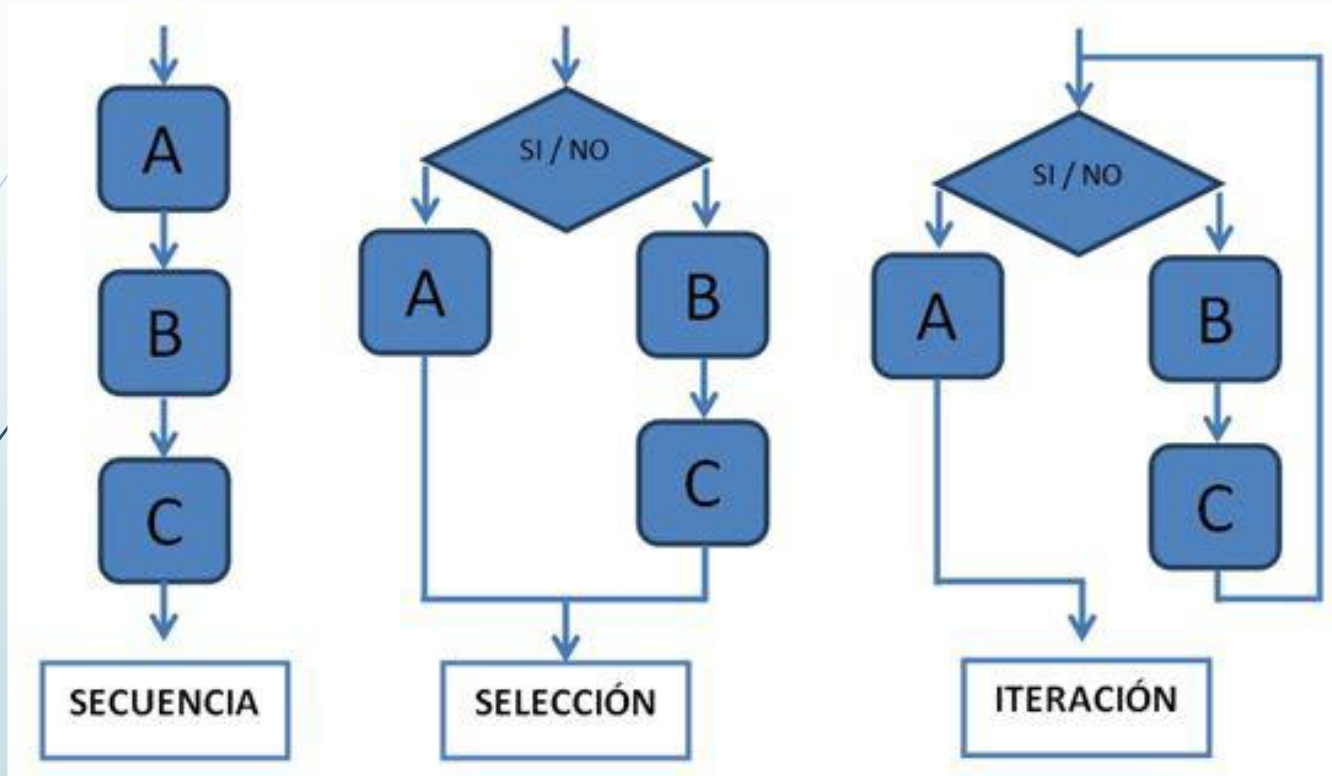


A diagram showing a rectangular box with the word "Strings" inside. The box is enclosed by a double quote " " symbol, indicating a string variable.

Tipos de estructuras en PHP

- Selección
 - Decidir acciones según condiciones
 - if, else, elseif, switch
- Iteración
 - Repetir acciones y generar bucles
 - for, while, do while, foreach

Tipos de estructuras en PHP

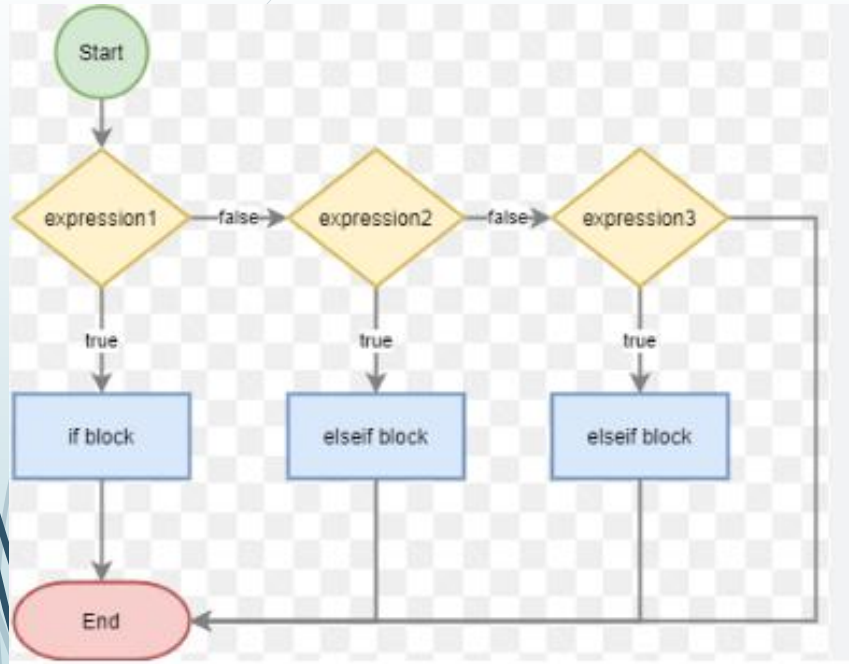


Selección : If /Else / Eself

- Si se cumple una condición se ejecutan instrucciones
- La evaluación devuelve TRUE/FALSE
- Es posible anidar
- Para comparar ==
- Es posible utilizar comparadores lógicos

Selección : If /Else / Elseif

Sintaxis:



```
<?php

$a = 5;

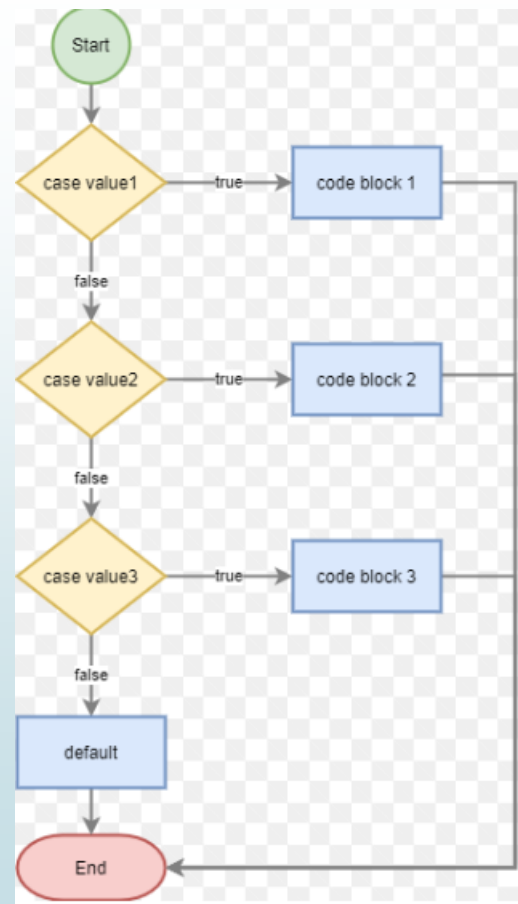
if ($a = 5):
    echo "a equals 5";
    echo "... ";
elseif ($a = 6):
    echo "a equals 6";
    echo "!!!";
else:
    echo "a is neither 5 nor 6";
endif;

?>
```

Selección: switch

- Comparar la misma variable con muchos valores
- Como muchos if
- Sintaxis

```
switch ( EXPRESSION )  
{  
    case 0: //code;  
    break;  
  
    case 1: //code;  
    break;  
  
    case 2: //code;  
    break;  
  
    default: //code;  
}
```



Iteración: for

- Repite un conjunto de instrucciones
- Un número conocido de veces
- Elementos
 - Valor inicial
 - Valor final
 - Incremento/Decremento

```
KEYWORD      (define counter)      (conditional expression)      (modify counter)
for ( $i = 0; $i < 10; $i++ )
{
    echo $i; //output: 0123456789
}
```

Iteración arrays: foreach

- Iteración sobre arrays/objetos

```
foreach ($array as $key => $value)
{
    //code;
}
```

```
$salmentak = ['Tomatea'=> 29, 'Patata' => 80, 'Marrubia'=>290];

foreach ($salmentak as $salgaia => $zenbat){
    echo "Saldutakoa:  $salgaia $zenbat  Kg <br>";
}
```

Saldutakoa: Tomatea 29 Kg
Saldutakoa: Patata 80 Kg
Saldutakoa: Marrubia 290 Kg

Iteración: while

- Repite un conjunto de instrucciones
- A diferencia de for se utiliza normalmente cuando no sabes exactamente cuántas veces se va a repetir.
- Se repite mientras se cumpla una condición.
- Elementos
 - Valor inicial
 - Valor final
 - Incremento/Decremento

```
<?php

$total = 0;
$number = 1;

while ($number <= 10) {
    $total += $number;
    $number++;
}

echo $total;
```

Control de flujo: break

- Finaliza la ejecución de la estructura de control

- for / foreach
- while / do while
- switch

```
for($j=1;$j<=10;$j++)
{
    echo "Valor de j: ".$j."<br/>";
    for($k=1;$k<=10;$k++)
    {
        echo "Valor de k: ".$k."<br/>";
        if( $k == 3) {
            //sale de los dos for
            break 2; //Probar tb sin el argumento
        }
    }
}
```

- Acepta un argumento

- Indica de cuantas estructuras salir

Valor de i: 1
Valor de i: 2
Valor de i: 3

Valor de j: 1
Valor de k: 1
Valor de k: 2
Valor de k: 3

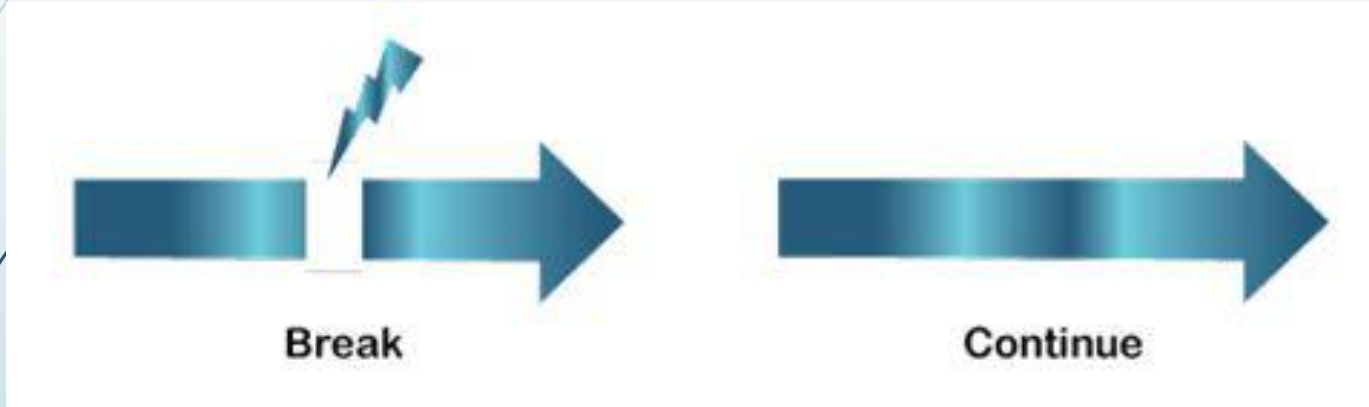
Control de flujo: continue

- Finaliza la iteración actual de la estructura de control e inicia una nueva iteración
- Acepta un argumento
 - Indica cuántos niveles de bucles se ha de saltar

```
for($i=1;$i<=10;$i++)  
{  
    if($i == 3) {  
        continue;  
    }  
  
    echo "Valor de i: ".$i."<br/>";  
}
```

```
Valor de i: 1  
Valor de i: 2  
Valor de i: 4  
Valor de i: 5  
Valor de i: 6  
Valor de i: 7  
Valor de i: 8  
Valor de i: 9  
Valor de i: 10
```

Control de flujo



INCLUDE

- Incluye y evalúa el archivo especificado
 - Embebe el código de un fichero PHP en otro fichero

PHP

- Si no se encuentra muestra un Warning
- `include_once`



■ Si el código del fichero ya ha sido incluido no se vuelve a incluir

REQUIRE

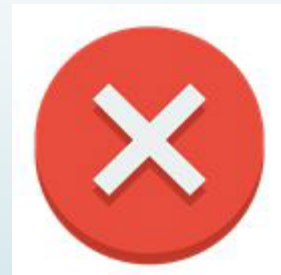
- Incluye y evalúa el archivo especificado
 - Embebe el código de un fichero PHP en otro fichero

PHP

- Si no se encuentra muestra un error fatal
- `require_once`

- Si el código del fichero ya ha sido incluido no se

vuelve a incluir



GOTO

- Para saltar a otra sección del código
 - No recomendable

Formularios Conceptos

- Permite al usuario introducir datos estructurados
- Los datos recogidos son almacenados y procesados
- Se complementan con HTML
 - Se verifican en el cliente mediante JavaScript
 - Se verifican en el servidor mediante PHP

Formularios acceso a datos

- Mediante un array asociativo
- Depende del método de envío definido en HTML

- \$_GET

- Límite datos a enviar
- En la URL
- Eficiente

- \$_POST

- Ficheros
- Mas seguro

The image shows a web form titled "FORMULARIO ESTA - Información personal" with a small US flag icon. The form contains several input fields and labels:

- Indicar su sexo:** Radio buttons for "Hombre" and "Mujer".
- Apellido:** A text input field with the placeholder "Apellido (como se muestra en su pasaporte)".
- Nombre:** A text input field with the placeholder "Solo nombre".
- Fecha de nacimiento:** Three input fields for "Día", "Mes", and "Año", each with a dropdown arrow.
- Ciudad de nacimiento:** A text input field.
- País de nacimiento:** A dropdown menu with "Spain" selected.
- Nacionalidad:** A dropdown menu with "Spain" selected.
- Número de pasaporte:** A text input field.
- País donde ha sido emitido su pasaporte:** A dropdown menu with "Spain" selected.
- Fecha de emisión del pasaporte:** Three input fields for "Día", "Mes", and "Año", each with a dropdown arrow.
- Fecha de vencimiento de su pasaporte:** Three input fields for "Día", "Mes", and "Año", each with a dropdown arrow.

Formularios ejemplos

```
<form action="welcomeGet.php" method="get">
  aaaaa: <input type="text" name="izena"><br><br>
  E-mail: <input type="text" name="emaila"><br><br>

  <input value="bidali" type="submit">
</form>
```

Formulario HTML

```
<?php
if (isset($_GET["izena"])){
    echo "Ongi etorri ".$_GET["izena"]."<br>";
};

if (isset($_GET["emaila"])){
    echo "Ongi etorri ".$_GET["emaila"]."<br>";
};
?>
```

PHP welcomeGet.php

Formularios ejemplos

```
<form action="welcomePost.php" method="post">
  Nombre: <input type="text" name="izena"><br><br>
  E-mail: <input type="text" name="emaila"><br><br>

  <input value="enviar" type="submit">
</form>
```

Formulario HTML

```
<?php
if (isset($_POST["izena"])){
    echo "Ongi etorri " . $_POST["izena"] . "<br>";
};

if (isset($_POST["emaila"])){
    echo "Ongi etorri " . $_POST["emaila"] . "<br>";
};
?>
```

PHP welcomePost.php

Formularios acceso a datos

- Determinar si una variable está vacía
 - empty()

```
empty ( mixed $var ) : bool
```

- Determinar si una variable está definida
 - isset()

```
isset ( mixed $var [, mixed $... ] ) : bool
```

Acceso a datos por isset (s)

En PHP el operador es un **null coalescing operator** que:

- Si la variable existe y no es null devuelve su valor.
- En caso contrario devuelve el valor por defecto especificado por usted.

Ejemplo:

```
<?php
// $_GET['izena'] bada go eta null ez bada, hori erabiliko da
// Bestela, "Gonbidatua" izango da balioa
$izena = $_GET['izena'] ?? "Gonbidatua";

echo "Kaixo, $izena!";
?>
```

Acceso a datos ejemplos

Balioa (`$var = ...`)

`isset($var)` emaitza

`empty($var)` emaitza

Azalpena

`null`

✗ false

✓ true

`isset` false da, NULL baitago; `empty` true da, hutsatzat hartzen duelako.

`0`

✓ true

✓ true

Aldagaia existitzen da, baina 0 `empty` -rentzat huts da.

`"0"` (string)

✓ true

✓ true

Stringa definituta dago (`isset` = true), baina `"0"` ere hutsatzat hartzen da.

`""` (string hutsa)

✓ true

✓ true

Existitzen da, baina string hutsa da.

`false`

✓ true

✓ true

Existitzen da, baina boolean false `empty` -rentzat huts da.

`true`

✓ true

✗ false

Existitzen da, eta balioa huts ez denez, `empty` false.

`array()` (hutsa)

✓ true

✓ true

Array hutsa hutsatzat hartzen du.

`array(1,2,3)`

✓ true

✗ false

Array-a ez dago hutsik, beraz `empty` false.

`"test"`

✓ true

✗ false

Existitzen da eta ez da hutsik.

aldagaia ez definitua

✗ false

✓ true

`isset` false, ez dagoelako; `empty` true, huts bezala tratatzen duelako.

Acceso a datos

- Puedes utilizar **isset ()** para asegurar que la variable está definida y disponible.
- Puedes utilizar **empty ()** para comprobar si la variable tiene o no valores útiles.

Buenas prácticas en isset ()

- Se utiliza **&&** para ir comprobando las condiciones individualmente.
- Si uno no se cumplimenta los otros no se comprobarán.
- Muy útil en el caso de isset (). Porque si no está habilitado no se generará el error.

```
if (isset($_POST["izena"])&& $_POST["izena"]=="Oier"){  
    echo "Ongi etorri ".$_POST["izena"]."<br>";  
};
```

Acceso a Datos - CheckBox (cada uno con su name)

```
<form method="post" action="checkboxName.php">
  <label>
    <input type="checkbox" name="sagarra" value="sagarra"> Sagarra
  </label><br>
  <label>
    <input type="checkbox" name="laranja" value="laranja"> Laranja
  </label><br>
  <label>
    <input type="checkbox" name="bananak" value="bananak"> Bananak
  </label><br>
  <input type="submit" value="Bidali">
</form>
```

Formulario HTML

Acceso a Datos - CheckBox (cada uno con su name)

```
if (isset($_POST['sagarra'])) {  
    echo "Aukeratu duzu: " . htmlspecialchars(string: $_POST  
        ['sagarra']) . "<br>";  
}  
if (isset($_POST['laranja'])) {  
    echo "Aukeratu duzu: " . htmlspecialchars(string: $_POST  
        ['laranja']) . "<br>";  
}  
if (isset($_POST['bananak'])) {  
    echo "Aukeratu duzu: " . htmlspecialchars(string: $_POST  
        ['bananak']) . "<br>";  
}  
  
// Inork hautatu ez badu:  
if (!isset($_POST['sagarra']) && !isset($_POST['laranja']) && !isset  
    ($_POST['bananak'])) {  
    echo "Ez duzu ezer aukeratu.";  
}
```

PHP

checkboxName.php

Acceso a Datos - CheckBox

(cada uno con su name)

htmlspecialchars(): Convierte caracteres especiales de un texto en caracteres que HTML mostrará como texto, en lugar de interpretarlos como código HTML.

Ejemplo

```
$texto = "<h1>Hola</h1>";  
  
echo $texto;
```

Acceso a Datos - CheckBox (cada uno con su name)

El navegador interpreta `<h1>` como una etiqueta HTML, por lo que veremos:

Hola

Pero nosotros quizá queremos mostrar literalmente:

`<h1>Hola</h1>`

Ahí utilizamos `htmlspecialchars()`:

```
$texto = "<h1>Hola</h1>";
```

```
echo htmlspecialchars($texto);
```

Acceso a Datos - CheckBox (cada uno con su name)

¿Qué ha ocurrido?

PHP ha convertido:

< → <

> → >

Internamente queda:

<h1>Hola</h1>

El navegador interpreta < como el carácter <, pero **no como una etiqueta HTML.**

Acceso a Datos - CheckBox (con arrays en la name)

```
<form method="post" action="checkboxArray.php">
  <label>
    <input type="checkbox" name="frutak[]" value="sagarra">
    Sagarra
  </label><br>
  <label>
    <input type="checkbox" name="frutak[]" value="laranja">
    Laranja
  </label><br>
  <label>
    <input type="checkbox" name="frutak[]" value="bananak">
    Bananak
  </label><br>
  <input type="submit" value="Bidali">
</form>
```

Formulario HTML

Acceso a Datos - CheckBox (con arrays en la name)

```
if (!empty($_POST['frutak'])) {  
    foreach ($_POST['frutak'] as $fruta) {  
        echo "Aukeratu duzu: " . htmlspecialchars(string: $fruta) .  
            "<br>";  
    }  
} else {  
    echo "Ez duzu ezer aukeratu."  
}
```

PHP

checkboxArray.php

Formulario Acceso a Datos

- En PHP, **filter_input()** sirve para obtener un dato que viene de una entrada externa. Por ejemplo, de un formulario GET o POST y, opcionalmente, validarlo o filtrarlo.
- **filter_input()** = recoger un dato de `$_GET`, `$_POST`, etc. aplicándole un filtro.

Sintaxis

```
filter_input(tipo, nombre, filtro);
```

Formulario Acceso a Datos

Ejemplo

```
$nombre = filter_input(INPUT_GET, "nombre", FILTER_SANITIZE_SPECIAL_CHARS);
```

`FILTER_SANITIZE_SPECIAL_CHARS` sirve para **limpiar un texto convirtiendo determinados caracteres especiales en entidades HTML**, de forma que puedan mostrarse como texto en una página HTML y no se interpreten como etiquetas.

Esto busca un parámetro llamado **nombre** dentro de:

```
$_GET["nombre"]
```

pero aplicando el filtro indicado.

Formulario Acceso a Datos

Ejemplo

Si tenemos esta URL:

```
http://localhost:8000/index.php?nombre=Laura
```

Podemos hacer:

```
$nombre = filter_input(INPUT_GET, "nombre");  
  
echo $nombre;
```

El resultado es:

Laura

Formulario Acceso a Datos

Constante	Origen
INPUT_GET	\$_GET
INPUT_POST	\$_POST
INPUT_COOKIE	\$_COOKIE
INPUT_SERVER	\$_SERVER
INPUT_ENV	\$_ENV

Formulario Acceso a Datos

Ejemplo- Filtrar un número

```
$edad = filter_input(INPUT_GET, "edad", FILTER_VALIDATE_INT);  
  
if ($edad === false) {  
    echo "La edad no es válida";  
} else {  
    echo "Edad: " . $edad;  
}
```

Formulario Acceso a Datos:

VALIDATE vs SANITIZE

Validate responde a la pregunta: "¿El dato cumple las condiciones?"

```
$email = filter_input(  
    INPUT_POST,  
    "email",  
    FILTER_VALIDATE_EMAIL  
);
```

- Si llega lorea@gmail.com devuelve el email.
- Si llega lorea devuelve false.

Formulario Acceso a Datos:

VALIDATE vs SANITIZE

Sanitize responde a la pregunta: "¿Puedo limpiar/modificar este dato para eliminar determinados caracteres? "

```
$nombre = filter_input(  
    INPUT_POST,  
    "nombre",  
    FILTER_SANITIZE_SPECIAL_CHARS  
);
```